Cynbit LeadIQ

Technical Approach Report

Prototype architecture, implementation, evaluation, and limitations

Prepared for: Cynbit Technologies | Assessment prototype | 16 October 2026

Implementation status
The Streamlit interface runs locally. The priority-scoring test suite currently reports 3 passing tests. During the demonstrated inquiry, Gemini returned a service error and the app successfully displayed a labelled local heuristic fallback. Live Gemini analysis has not been verified as successful.

1. Scope and requirements

LeadIQ is a single-user proof of concept for turning free-text prospective-customer inquiries into structured qualification data and a suggested response. The scope deliberately avoids custom model training, multi-agent orchestration, CRM writes, and outbound email.

Functional requirements

- Accept a text inquiry and provide three built-in sample inquiries.
- Classify into AI Automation, Software Development, CRM Integration, Document Processing, Voice AI, Other, or Unclear.
- Extract business goal, stated budget, timeline, integrations, and missing information.
- Calculate a 0–10 priority score and return a High/Medium/Low label with reasons.
- Produce an editable and downloadable response draft.
- Clearly identify whether output came from live Gemini or demo fallback.

Non-functional requirements

- Validate structured output with Pydantic.
- Reject empty inquiries and inputs above 8,000 characters.
- Keep API credentials in a local environment file excluded from source control.
- Fail safely: no external email or CRM actions; require human review.

2. Architecture

Layer	Responsibility
Streamlit UI (app.py)	Inquiry input, sample selection, result display, draft editing and download.
Pipeline (src/pipeline.py)	Input validation, Gemini request, output parsing, fallback handling, and orchestration.
Schema (src/schemas.py)	Pydantic models and constrained categories, priority labels, and scores.
Prompt + knowledge base	System instructions and a small, explicitly limited set of service descriptions.
Priority rules	Deterministic score and reasons based on available qualification signals.
Tests and sample data	Unit tests for scoring and synthetic inquiries for manual demonstrations.

Processing flow

Inquiry → validation → Gemini structured extraction attempt → Pydantic validation → deterministic priority scoring → Streamlit review screen. If the Gemini request or parsing fails, the pipeline uses local keyword heuristics and marks the result as “Demo fallback.”

3. Data model and scoring

LeadExtraction holds service_category, business_goal, optional budget and timeline, integrations, missing_information, and draft_reply. LeadAnalysis extends it with priority, priority_score (0–10), priority_reasons, analysis_mode, and an optional analysis_note.

The priority score is generated by the existing Python function calculate_priority(), not by the LLM. The current demonstration produces High / 10 out of 10 for the sample inquiry because the rule engine recognizes a service category, budget, timeline, and integration. This is an illustrative heuristic result, not a statistically calibrated likelihood of winning the lead.

Fallback extraction uses keyword matching and a regular expression for a limited set of budget and timeline formats. It is intentionally simple: ambiguous wording, uncommon currency formats, synonyms, or nuanced business goals

may be misclassified. Missing-information checks are also heuristic.

4. Reliability and security

- Keep GEMINI_API_KEY in .env; never hard-code it or commit it.
- Keep .env in .gitignore and commit only .env.example with placeholder values.
- Treat customer inquiries as untrusted content and constrain the LLM to the approved context.
- Validate structured output before displaying it. If parsing fails, use the explicit fallback path.
- Avoid logging full inquiries or secrets. Production logging should retain only necessary diagnostic metadata.
- Do not send messages or write to CRM automatically; the prototype only presents and downloads a draft.
- A broad exception fallback improves demo continuity but can hide distinct failure causes from the user. Production code should log categorized error types and expose safe diagnostics.

5. Test results and evaluation plan

Observed automated tests: 3 passed, 0 failed in 0.03 seconds on Windows with Python 3.14.3 and pytest 9.1.1. Tests shown in the terminal output: complete lead gets high priority; missing details get lower priority; score is never above ten. These tests cover the scoring rules only; they do not establish extraction accuracy, live Gemini reliability, or response quality.

Recommended additional test cases

- Timeline parser recognizes “in four weeks” and a numeric form such as “within 3 days.”
- Budget and timeline are marked missing when not stated.
- An unrelated inquiry maps to Other or Unclear rather than a confident unsupported category.
- Empty input and inputs above 8,000 characters are rejected.
- Gemini timeout, 503, invalid response, and missing key trigger a labelled fallback.
- Prompt-injection text inside an inquiry cannot change system instructions or produce unsupported promises.
- Draft output contains no invented price, delivery guarantee, or fabricated client reference.

Evaluation metrics for a future labelled set

- Service-category accuracy and per-category precision/recall.
- Field-level extraction accuracy for budget, timeline, integrations, and business goal.
- Missing-field detection precision/recall.
- Human rating of reply relevance, groundedness, tone, and edit effort.
- Live request success rate, latency, and cost per analysis.

No accuracy, latency, cost, or business-impact metrics are claimed because no labelled benchmark or successful live-model run has been recorded.

6. Limitations and future work

- The knowledge base is a small prototype context, not a complete or independently verified service catalogue.
- The fallback is rule-based and should not be marketed as equivalent to generative AI.
- No persistent database, user authentication, role-based access, audit history, or CRM integration is implemented.
- No production deployment, load testing, privacy review, or security review has been completed.
- Future phases: labelled evaluation set, improved parser and confidence indicators, persistent storage with access controls, human approval workflow, and a sandbox CRM integration.

Conclusion

The prototype demonstrates the core interaction and a resilient local fallback. The current evidence supports a functional demonstration and passing score-rule tests, not production readiness or validated AI performance. The next milestone should be evaluation and reviewer feedback before integrating external systems.

References

[1] Cynbit Technologies services, https://www.cynbit.in/services (accessed 16 October 2026).

[2] Google Gemini API documentation, https://ai.google.dev/gemini-api/docs (consulted for integration guidance).